

Revision	Date	Updates	Approved By	Version_No.	Link to the SRS
Initial Version	January 20th 2026	Initial SRS document	Team	1.0	SRS_1.0
First Revision	January 21st 2026	Added REQ-TX-002 and REQ-TX-003.	Arnav	1.1	
		Added functional requirements and non-functional requirements.			
		Updated architecture to follow sharding pattern.			
Second Revision	January 24th 2026	Added REQ-REP-003 and REQ-REP-004 (failure detection and primary failover).	Team/ Reviewer Feedback	1.2	
		Added Requirements priorities.			
		Added Requirements priorities.			
		Naming conventions templates were documented.			
Third Revision	January 26th 2026	Added write-ahead logging requirements (REQ-DATA-002).	Subhanshu	1.3	
		Created the following new diagrams: timing, component.			
		Documented WAL recovery semantics to ensure atomicity.			
Fourth Revision	January 30th 2026	Added Kafka integration (REQ-COORD-001).	Anarghya	2.0	
		Mapped FR and NFR to sprints			
		Defined stateless coordinator model.			
Fifth Revision	February 4th 2026	Added load balancing requirements (REQ-DATA-003).	Team/ Reviewer Feedback	2.1	
		Revised the model for deadlock issue handling.			
		Documented shard rebalancing strategy.			
Sixth Revision	February 10th 2026	Added performance requirements (REQ-PERF-001 to REQ-PERF-004).	Arnav	2.2	
		Modified the following UML diagrams			
		Defined latency targets.			
Seventh Revision	February 15th 2026	Modified UML diagrams: Class, Activity and Use_Case.	Satwik	3.0	SRS_3.0
Eight Revision	February 18th 2026	Updated Component Responsibilities.	Subhanshu	3.1	

Date	Diagram Category	Updates	Version_No.	Approved by	Link
13/2/26	Class Diagram	Separated single-shard and 2PC logic clearly. Added PartitionManager and Recovery inside ShardServer. Standardized transaction states and operation types.	2	Anarghya	Timing_Diagram_version_2.png
15/2/26	Sequece Diagram	Added quorum acknowledgment. Aligned method names with class diagram.	2	Satwik	Composite_Diagram_Version_2.png
19/2/26	Component Diagram	Organized into layered architecture. Added Engine abstraction for storage. Clarified WAL-based replication flow.	2	Arnav	Class_Diagram_Version_2
20/2/26	Composite Diagram	Expanded internal structure of API Gateway and Shard Node. Separated primary and follower responsibilities. Clarified internal wiring.	2	Satwik	Sequence_Diagram_Version_2.png
24/2/26	Activity Diagram	Clearly separated single-shard and 2PC flows. Added decision points and crash recovery steps.	2	Subhanshu	Component_Diagram_Version_2.png
27/2/26	Use-Case Diagram	Refined actor roles. Added status polling and monitoring use cases. Clarified authentication linkage.	2	Anarghya	Activity_Diagram_Version_2.png
1/3/26	Timing Diagram	Separated single-shard and 2PC timelines. Highlighted prepare, commit, and replication phases.	2	Arnav	Deployment_Diagram_Version_2.png
2/3/26	Deployment Diagram	Separated API, coordinator, and shard nodes. Explicitly modeled primary, followers, and message broker.	2	Subhanshu	Use_case_diagram_version_2.png

	Requirements (Functional / Non-functional / Business)										Test / Integration Status Details					
Attribute	Component	Class	Functions	Start Date of Code	End Date of Code	Update Date	Update Made	Reason for Update	Team Member(s) Designing / Updating Code	Team Member(s) Testing Code	Complete / Ongoing / Abandoned	Ticket Ids raised	Date of Ticket Generation	Date of Ticket Resolution	Link	
Description	These codes come from from the SRS sections	The component name that solves the requirements	The list of classes within the component that apply to the requirements	[mm/dd/yyyy]			These can be code changes / integrations / etc			These fields will receive data from the Ticket table						
Examples	REQ-TX-001	Ledger	Ledger	NewLedger; ApplyDebit; ApplyCredit; GetBalance; ValidateVariants	1/15/2026	1/28/2026	2/1/2026	Bug fix in concurrent access	Thread-safety improvement	Neeli Satwik; Anarghya Das					https://github.com/Subhanshu2406/Software-Course-Project/blob/5506853bea87c9aa96017b974882a3f689bc070/shard/ledger/ledger.go	
			Transaction	Transaction; NewTransaction	1/15/2026	1/20/2026									https://github.com/Subhanshu2406/Software-Course-Project/blob/5506853bea87c9aa96017b974882a3f689bc070/shard/models/transaction.go	
	REQ-DATA-001	Partition	PartitionMapper	Hash_GetPartition; GetPartitionID	2/1/2026	2/10/2026	2/15/2026	Enhanced hash distribution	Better load balancing	Anarghya Das; Anav Vivek					https://github.com/Subhanshu2406/Software-Course-Project/blob/5506853bea87c9aa96017b974882a3f689bc070/shard/ultra/hash.go	
			PartitionManager	NewPartitionManager; OwnsPartition; AddPartition; RemovePartition	2/5/2026	2/12/2026									https://github.com/Subhanshu2406/Software-Course-Project/blob/5506853bea87c9aa96017b974882a3f689bc070/shard/partition/partition.go	
	REQ-SAFE-001	Write Ahead Logging	WAL	NewWAL; Append; Fsync; ReadFrom; Truncate	1/20/2026	2/5/2026	2/20/2026	Added checkpointing support	Performance optimization	Anav Vivek; Neeli Satwik					https://github.com/Subhanshu2406/Software-Course-Project/blob/5506853bea87c9aa96017b974882a3f689bc070/shard/wal/wal.go	
			WALEntry	WALEntry; NewWALEntry	1/20/2026	1/25/2026									https://github.com/Subhanshu2406/Software-Course-Project/blob/5506853bea87c9aa96017b974882a3f689bc070/shard/models/wal_entry.go	
	REQ-SAFE-002	Recovery	RecoveryEngine	Recover; AccumulateBalances; ReplayCommitted; ValidateState	2/1/2026	2/18/2026	2/25/2026	Fixed zero- balance debit bug	Net-balance accumulation strategy	Subhanshu Gupta; Anarghya Das					https://github.com/Subhanshu2406/Software-Course-Project/blob/5506853bea87c9aa96017b974882a3f689bc070/shard/recovery/recovery.go	
			Ledger	SetBalance; GetBalance	2/8/2026	2/12/2026									https://github.com/Subhanshu2406/Software-Course-Project/blob/5506853bea87c9aa96017b974882a3f689bc070/shard/ledger/ledger.go	
	REQ-HA-001	Replication	PrimaryReplicator	NewPrimaryReplicator; Replicate; WaitForQuorumACK; SendToFollowers	2/15/2026	3/2/2026	3/2/2026	Integrated quorum ACK logic	High availability implementation	Subhanshu Gupta; Anav Vivek					https://github.com/Subhanshu2406/Software-Course-Project/blob/5506853bea87c9aa96017b974882a3f689bc070/shard/replication/primary.go	
			FollowerReceiver	NewFollowerReceiver; ReceiveEntry; PersistWAL; SendACK	2/18/2026	2/28/2026									https://github.com/Subhanshu2406/Software-Course-Project/blob/5506853bea87c9aa96017b974882a3f689bc070/shard/replication/follower.go	
			ReplicationManager	ReplicationMgr; Coord M anagerReplicas; Monitor	2/20/2026	3/2/2026									https://github.com/Subhanshu2406/Software-Course-Project/blob/5506853bea87c9aa96017b974882a3f689bc070/shard/replication/client.go	

Date	Alpha Concern	Alpha Card	Current Card	Status	Notes to support status	Approved by
01/01/2026	Customer	Stakeholders	Recognized	1/6	Instructor and team identified as primary stakeholders.	Arnav Vivek
15/01/2026	Customer	Stakeholders	Represented	2/6	Roles and responsibilities agreed within team.	Arnav Vivek
02/02/2026	Customer	Stakeholders	Involved	3/6	Instructor feedback incorporated during SRS v2.0 revision.	Arnav Vivek
05/01/2026	Customer	Opportunity	Identified	1/6	Need for scalable distributed ledger system defined.	Neeli Satwik
18/01/2026	Customer	Opportunity	Solution Needed	2/6	Sharding and 2PC selected as architectural approach.	Neeli Satwik
02/02/2026	Customer	Opportunity	Value Established	3/6	Scope, constraints and objectives formalized in SRS v2.0.	Neeli Satwik
10/01/2026	Solution	Requirements	Conceived	1/6	Initial functional requirements drafted.	Anarghya Das
22/01/2026	Solution	Requirements	Bounded	2/6	Requirements categorized (TX, DATA, REP, COORD).	Anarghya Das
02/02/2026	Solution	Requirements	Coherent	3/6	UML diagrams and traceability matrix completed.	Anarghya Das
25/01/2026	Solution	Software System	Architecture Selected	3/6	Gateway, Coordinator, Shard architecture finalized.	Anarghya Das; Subhanshu Gupta
14/02/2026	Solution	Software System	Demonstrable	4/6	WAL-backed single-shard execution completed.	Anarghya Das; Subhanshu Gupta
28/02/2026	Solution	Software System	Demonstrable	4/6	Multi-shard routing working end-to-end.	Anarghya Das; Subhanshu Gupta
01/01/2026	Endeavor	Team	Seeded	1/5	Team members confirmed.	Arnav Vivek
10/01/2026	Endeavor	Team	Formed	2/5	Sprint roles assigned.	Arnav Vivek
20/02/2026	Endeavor	Team	Collaborating	3/5	Coordinated sprint execution across modules.	Arnav Vivek
01/02/2026	Endeavor	Work	Initiated	1/6	Sprint 1 tasks started.	Neeli Satwik
15/02/2026	Endeavor	Work	Prepared	2/6	Sprint 2 backlog defined.	Neeli Satwik
02/03/2026	Endeavor	Work	Under Control	3/6	Sprint 3 started; cross-shard coordination in progress.	Neeli Satwik

05/01/2026	Endeavor	Way of Working	Principles Established	1/6	Sprint structure and coding conventions defined.	Subhanshu Gupta
20/01/2026	Endeavor	Way of Working	Foundation Established	2/6	Tools, repo structure, and workflow formally set up.	Subhanshu Gupta
15/02/2026	Endeavor	Way of Working	In Use	3/6	Structured sprint workflow actively followed.	Subhanshu Gupta

Category	Expected Start Date	Expected End Date	Actual Start Date	Actual End Date	Percent Progression	Sprint Manager	Description
Project Planning	1/12/2026	1/14/2026	2/12/2026	2/14/2026	100%	N/A	Project opportunity identification and value establishment. The initial requirements are finalized.
Requirements and SRS	1/14/2026	2/4/2026	1/14/2026	N/A	75%	N/A	The requirements part will keep evolving in the agile methodology
Sprint 1	2/26/2026	3/4/2026	1/3/2026	5/3/2026	70%	Neeli Satwik	Establish the core system foundation: Database, Authentication and Sharding Logic.
Sprint 2	3/5/2026	3/11/2026	N/A	N/A	0%	Anarghya Das	Implement essential middleware such as WAL and pre-allocation features to handle the ACID conditions.
Sprint 3	3/12/2026	3/18/2026	N/A	N/A	0%	Subhanshu Gupta	Complete the fundamental load-balancing workflow and implement coordinator logic. Implement basic sharding requirements.
Sprint 4	3/19/2026	3/25/2026	N/A	N/A	0%	Arnav Vivek	Integrate advanced sharding requirements.
Sprint 5	3/26/2026	4/1/2026	N/A	N/A	0%	Neeli Satwik	Implement advanced notification and UI/UX, and perform quality checks.
Sprint 6	4/2/2026	4/8/2026	N/A	N/A	0%	Arnav Vivek	Final System Validation
Final Review	4/9/2026	4/15/2026	N/A	N/A	0%	N/A	N/A